國立臺北科技大學
National Taipei University of Technology

資料結構
Data Structure

Lab 06

姓名:夏宇柔
學號:113840005

Lab06-Q1.
Complete the following sections of the sample code: • **Create a class named 'Queue' that includes the methods: 'enqueue', 'dequeue', 'isEmpty', and 'isFull'.** • **Implement the 'initializeDeck' method.** • **Implement the 'initializePlayer' method.** • **Create the 'playerTurn' method.** • **Create the 'dealerTurn' method.**
Code

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
#include <ctime>
using namespace std;

struct Player { // 玩家結構體，包含名稱和分數
    string name; // 玩家名稱
    int score;   // 玩家分數
};

class Card { // 建立一個Card類別，用來表示單張撲克牌
public:
    string suit; // 花色
    string rank; // 點數
    Card() {}
    Card(string s, string r) : suit(s), rank(r) {}
    void display() const {
        cout << rank << " of " << suit << endl;
    }
};

class Deck { // 牌組類別，負責生成和洗牌
private:
    Card cards[52]; // 52 張牌
    int index; // 目前發牌索引
public:
    Deck() : index(0) { // 初始化牌組
        string suits[] = { "Hearts", "Diamonds", "Clubs", "Spades" }; // 四種花
色
        string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "J",
"Q", "K", "A" }; // 13 張牌
        int k = 0;
        for (int i = 0; i < 4; i++) { // 4 種花色
            for (int j = 0; j < 13; j++) { // 13 種點數
                cards[k] = Card(suits[i], ranks[j]);
                k++;
            }
        }
    }
    void shuffleDeck() { // 洗牌
        srand(time(0));
        for (int i = 51; i > 0; i--) { // 從最後一張牌開始，向前遍歷
            int j = rand() % (i + 1); // 生成隨機索引 j
```

```
            swap(cards[i], cards[j]); // 交換牌組中的兩張牌
        }
    }
    // 取得指定位置的牌
    Card getCard(int i) const {
        return cards[i]; // return card at position i
    }
};

// Queue 類別
class Queue {
private:
    vector<Card> deck; // 用來存放卡牌的容器
    int Front;         // 指向牌堆的前端
    int Rear;          // 指向牌堆的尾端
    int capacity;      // 容器最大容量
public:
    // 建構函式
    Queue(int size) : Front(-1), Rear(-1), capacity(size) {}
    // 插入元素到佇列尾端
    void enqueue(Card card) {
        if (isFull()) { // 檢查是否滿
            cout << "牌堆已滿，無法插入！" << endl;
            return;
        }
        if (Front == -1) {
            Front = 0;
        }
        deck.push_back(card); // 插入元素到尾端
        Rear++;               // 尾端指標移動
    }
    // 從佇列中移除最前端元素
    void dequeue() {
        if (isEmpty()) {                                    // check if empty
            cout << "Deck is empty! Cannot remove." << endl; // print error
            return;                                         // stop function
        }

        Front++; // move front pointer forward

        // if all elements removed
        if (Front > Rear) {
            Front = -1;   // reset front
            Rear = -1;    // reset rear
            deck.clear(); // clear vector
        }
    }
    Card front() { // 獲取佇列最前端元素
        if (isEmpty()) { // 檢查是否空
            cout << "牌堆為空，無法獲取最前端的牌！" << endl;
            return Card("", "");
        }
        return deck[Front];
    }
    bool isEmpty() const { // 判斷佇列是否為空
        // 提示:front和rear的關係，並且有兩個情況要檢查(用&&)
        return (Front == -1 && Rear == -1);
```

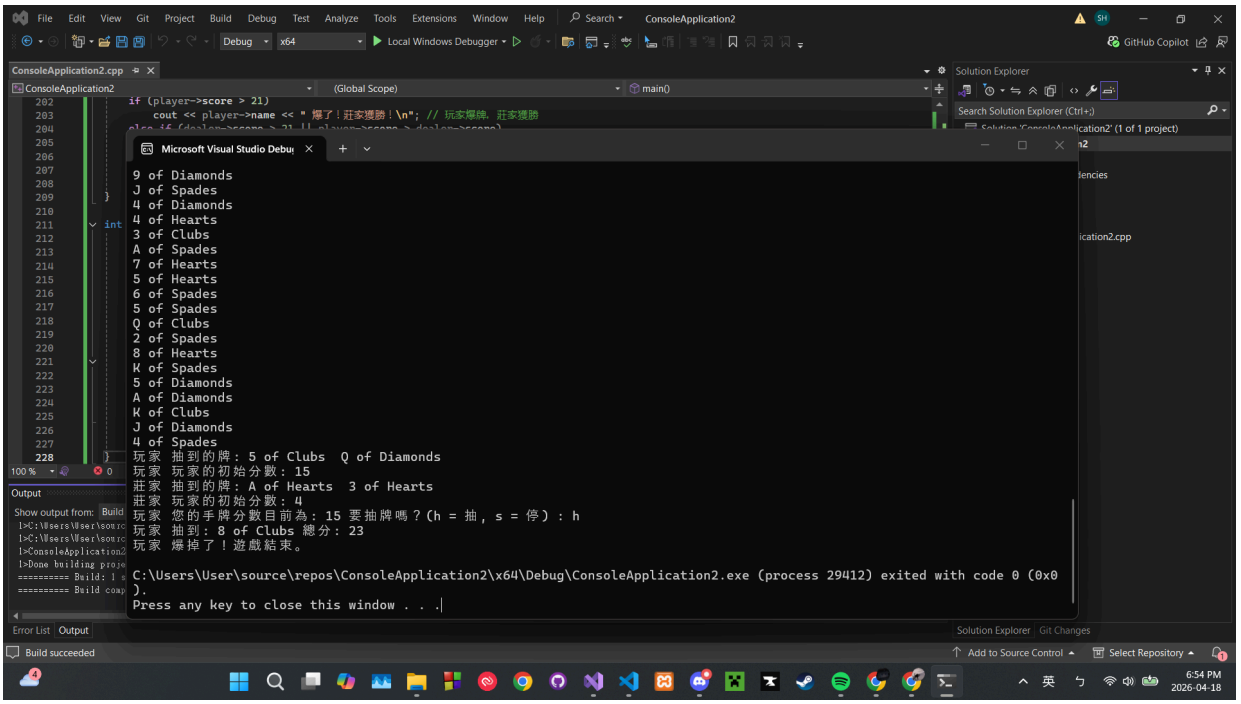
```
    }
    bool isFull() const { // 判斷佇列是否已滿
            // 提示:deck的大小和capacity的關係
        return deck.size() == capacity; // compare size with capacity
    }
};

void initializeDeck(Queue& cardDeck) { // 初始化並洗牌
    Deck deck; // create deck object
    cout << "初始牌堆: \n";
    for (int i = 0; i < 52; i++) { // loop through all cards
        deck.getCard(i).display(); // 印出原始牌堆
    }
    deck.shuffleDeck(); // 洗牌
    cout << "\n洗牌後的牌堆: \n";
    for (int i = 0; i < 52; i++) {
        deck.getCard(i).display(); // 印出洗牌後的牌堆
    }
    // 提示:將洗好的牌放入牌堆
    // 使用for迴圈, 將洗好的牌(deck.getCard(i))放入牌堆
    for (int i = 0; i < 52; i++) { // push all cards into queue
        cardDeck.enqueue(deck.getCard(i));
    }
}

void initializePlayer(Player* player, string name, Queue& cardDeck) { // 初始化
玩家, 發兩張牌
    player->name = name; // 玩家名稱
    player->score = 0; // 玩家分數
    cout << player->name << " 抽到的牌: ";

    for (int i = 0; i < 2; i++) { // 發兩張牌
        // 提示:從牌堆中取出一張牌, 然後從牌堆中移除這張牌
        // 注意:卡牌變數的類型是Card, 並且使用front抽牌
        Card drawnCard = cardDeck.front(); // get front card
        cardDeck.dequeue();                // remove it from queue

        // 將抽到的牌印出
        cout << drawnCard.rank << " of " << drawnCard.suit << "  ";

        if (drawnCard.rank == "A") player->score += 1; // 計算A分數
        else if (drawnCard.rank == "J" || drawnCard.rank == "Q" ||
drawnCard.rank == "K") player->score += 10; // 計算JQK分數
        else player->score += stoi(drawnCard.rank); // 計算2-10分數
    }
    cout << "\n";
    cout << player->name << " 玩家的初始分數: " << player->score << endl; // 印出
玩家的初始分數
}

void playerTurn(Player* player, Queue& cardDeck) { // 玩家回合
    char choice; // store user input
    while (player->score < 21 && !cardDeck.isEmpty()) { // 玩家小於21點且牌堆不為
空
        cout << player->name << " 您的手牌分數目前為: " << player->score << " 要抽
牌嗎?(h = 抽, s = 停) : "; // 顯示選擇
        cin >> choice; // 輸入選擇
```

```
        if (choice == 'h') { // 如果玩家選擇抽牌
            // 提示:從牌堆中取出一張牌, 然後從牌堆中移除這張牌
            // 注意:卡牌變數的類型是Card, 並且使用front抽牌
            Card newCard = cardDeck.front();  // get card
            cardDeck.dequeue();               // remove card

            if (newCard.rank == "A") player->score += 1; // 計算A點數
            else if (newCard.rank == "J" || newCard.rank == "Q" ||
newCard.rank == "K") player->score += 10; // 計算JQK點數
            else player->score += stoi(newCard.rank); // 計算2-10點數
            // 印出抽到的牌和總點數
            cout << player->name << " 抽到: " << newCard.rank << " of " <<
newCard.suit << " 總分: " << player->score << endl;
            if (player->score > 21) { // 如果玩家大於21點
                cout << player->name << " 爆掉了！遊戲結束。\n"; // 玩家爆牌, 莊家獲
勝
                return;
            }
        }
        else if (choice == 's') { // 如果玩家選擇停牌
            cout << player->name << " 選擇停牌, 總分: " << player->score << endl;
// 印出總點數
            break;
        }
        else {
            cout << "請輸入有效選項 (h = 抽, s = 停)！" << endl; // 輸入無效
        }
    }
}

void dealerTurn(Player* dealer, Queue& cardDeck) { // 莊家回合
    while (dealer->score < 17 && !cardDeck.isEmpty()) { // 莊家小於17點且牌堆不為
空
        // 提示:從牌堆中取出一張牌, 然後從牌堆中移除這張牌
        // 注意:卡牌變數的類型是Card, 並且使用front抽牌
        Card newCard = cardDeck.front();  // draw card
        cardDeck.dequeue();               // remove card

        if (newCard.rank == "A") dealer->score += 1; // 計算A點數
        else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank ==
"K") dealer->score += 10; // 計算JQK點數
        else dealer->score += stoi(newCard.rank); // 計算2-10點數
        // 印出抽到的牌和總點數
        cout << "莊家抽到: " << newCard.rank << " of " << newCard.suit << " 莊家
目前總分: " << dealer->score << endl;
        if (dealer->score > 21) { // 如果莊家大於21點
            cout << "莊家爆了！玩家獲勝！" << endl; // 玩家獲勝
            return;
        }
    }
}

// 判斷勝負
void determineWinner(Player* player, Player* dealer) {
    if (player->score > 21)
        cout << player->name << " 爆了！莊家獲勝！\n"; // 玩家爆牌, 莊家獲勝
```

```
    else if (dealer->score > 21 || player->score > dealer->score)
        cout << player->name << " 贏了！\n"; // 莊家爆牌或玩家點數大於莊家，玩家獲勝
    else if (player->score == dealer->score)
        cout << "平手！\n"; // 莊家與玩家點數相同，平手
    else cout << "莊家贏了！\n";
}

int main() {
    srand(time(0)); // seed random
    Queue cardDeck(52); // 宣告牌堆
    initializeDeck(cardDeck); // 初始化牌堆

    Player player, dealer; // 宣告並初始化莊家以及玩家
    initializePlayer(&player, "玩家", cardDeck); // deal to player
    initializePlayer(&dealer, "莊家", cardDeck); // deal to dealer

    playerTurn(&player, cardDeck); // player plays
    if (player.score <= 21) { // if player not bust
        cout << "\n莊家回合...\n";
        dealerTurn(&dealer, cardDeck); // dealer plays
        determineWinner(&player, &dealer); // decide winner
    }

    return 0;
}
```

Screenshots

Full Screen:



Output:

```
初始牌堆:
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
J of Clubs
Q of Clubs
K of Clubs
A of Clubs
2 of Spades
3 of Spades
4 of Spades
5 of Spades
6 of Spades
7 of Spades
8 of Spades
9 of Spades
10 of Spades
J of Spades
Q of Spades
K of Spades
A of Spades
```

```
玩家 抽到的牌: 5 of Clubs  Q of Diamonds
玩家 玩家的初始分數: 15
莊家 抽到的牌: A of Hearts  3 of Hearts
莊家 玩家的初始分數: 4
玩家 您的手牌分數目前為: 15 要抽牌嗎？(h = 抽, s = 停) : h
玩家 抽到: 8 of Clubs 總分: 23
玩家 爆掉了！遊戲結束。
```

Discussion Section

A queue follows the FIFO (First In First Out) principle, meaning the first card placed into the deck is the first one dealt, which matches how real card dealing works after shuffling. A stack follows LIFO (Last In First Out), where the last card added would be dealt first, reversing the order of the deck. Because Blackjack requires cards to be dealt in the same order as a shuffled deck, a queue is the correct structure to use, while a stack would produce an unrealistic dealing sequence.